

like `document.write` to display the results of a POST variable. The only real difference in the DOM-based vulnerability is that the server doesn't send back the results; instead, the DOM parses the code locally, and the malicious script is executed with the same privilege as the browser on the victim's machine. Consider a scenario where a vulnerable site has the following content (named, for example, `http://www.example.com/welcome.html`):

```
<HTML>
<SCRIPT>
var pos=document.URL.indexOf("name=")+5;
document.write(document.URL.substring(pos,document.URL.length));
</SCRIPT>
<BR>
Welcome to our site
-
</HTML>
```

Normally, the code in this page would welcome the user, if invoked with the following URL:

```
http://www.example.com/welcome.html?name=Joe
```

However, a little tampering with this URL results in displaying the users' cookies in their browser, if they click the URL hyperlink:

```
http://www.example.com/welcome.html?name=<script>alert(document.cookie)</script>
```

What happens is that to open this URL, the victim's browser sends an HTTP request to `www.example.com`. It receives the above (static) HTML page. The victim's browser then starts parsing this HTML into DOM. In this case, the code references `document.URL`, and so, a part of this string is embedded at parsing time in the HTML. It is then immediately parsed, and the malicious JavaScript code passed through the URL is executed in the context of the same page, resulting in an XSS attack.

You might realise here that the payload did arrive at the server (in the query part of the HTTP request), and so it could be detected just like any other XSS attack—but attackers even take care of that with something like the following:

```
http://www.example.com/welcome.html?name=<script>alert(document.cookie)</script>
```

Notice the hash sign (#) used here; it tells the browser that everything beyond it is a fragment, and not part of the query. IE (6.0) and Mozilla do not send the fragment to the server, and for these browsers, the server would only see `http://www.example.com/welcome.html`, with the payload remaining hidden.

Latest trends in XSS

Meta-information XSS (miXSS)

This new type of XSS vulnerability has emerged recently, and

exploits commonly used network administration utilities. It is found in those services that utilise valid user-provided input to gather data and display it for the user. It is in this data that the cross-site scripting occurs. Attackers can extract information about network administration utilities. Websites that allow you to perform DNS resolution, and websites that verify SPF records are more vulnerable to miXSS attacks. To learn more about miXSS, check the resources at the end of this article.

XSS Shell

The XSS Shell is a tool that can be used to set up an XSS channel between a victim and an attacker, so that an attacker can control a victim's browser, sending it commands. The communication is bi-directional.

XSS Tunnel

This is a GPL-licensed open source application written in .NET, and is the standard HTTP proxy which sits on an attacker's system. It enables tunnelling of HTTP traffic through an XSS channel, to use virtually any application that supports HTTP proxies.

XSS causing DDoS attacks

Recent trends have seen attackers using XSS-vulnerable sites as an initiator step in performing DDoS (Distributed Denial of Service) attacks. They trick users into downloading and installing plug-ins. When the user clicks the download link, besides the plug-in, a worm or bot (in most cases) gets installed in the background. This worm/bot can give the attacker full privileges over the user's system, and the attacker can then use it to perform DoS attacks, or to spread the botnet.

Botnets

A botnet is an alliance of interconnected computers infected with some malicious software agent (called a bot). Bots are commanded by an operator, and can typically be ordered to send spam mails, harvest information such as licence keys or banking data on compromised machines; or launch distributed denial-of-service (DDoS) attacks against arbitrary targets.

Securing a server against XSS

Cross-site scripting is a potential risk for most Web servers and browsers. Attackers are constantly coming up with new types of this attack, but the following best practices can help you secure your system against attackers.

For clients or users

Take a serious and suspicious view of e-mails or spam mail that contain big, bulky and suspicious URLs. Don't click such links, even if they are to known and trusted sites. Many of these messages try various tricks to coax you into clicking the link. Some of these include offers to make you financially strong and independent; others threaten that you will lose your account on